

Multiple Linear Regression — Variable Selection

Jericho Lawson

MATH 240

Week 6, Lecture #2 — Ch. 8

Model Selection

- Naturally, we might think that adding more variables can increase accuracy of predictions
 - ▶ Results in overfitting
 - ▶ Certain variables actually reduce accuracy
- Want to balance efficacy and size
 - ▶ Can prune variables that are less important
 - ▶ Optimized models through pruning are called **parsimonious**

Model Selection - Strategies

- **Backward elimination:** prune variables one-at-a-time, starting with the full model
 - ▶ The full model contains all available covariates that can explain the response
 - ▶ Prune variables until further removal is not statistically better
 - ★ e.g. improvement in adjusted R^2
- **Forward selection:** add variables one-at-a-time, starting with a model with no explanatory variables
 - ▶ Can add variables that increase accuracy of predictions
 - ▶ Keep adding variables until inclusion is not statistically better

Model Selection - Metrics (Summary)

- **Adjusted R^2 :** want to find model with higher response variation explained by the variation in the covariates
 - ▶ *Backward*: remove predictor associated with model (not containing predictor) with highest adjusted R^2 value in each round until we have not further improved the adjusted R^2 value
 - ▶ *Forward*: add predictor associated with model (not containing predictor) with highest adjusted R^2 value in each round until we have not further improved the adjusted R^2 value

Tutorial: Backward Elimination w/ R^2

Recall the original model fit:

- `mod1 = lm(price ~ bed + bath + area + year_built + cooling + lot, data = duke_forest)`
- `summary(mod1)`

Call:

```
lm(formula = price ~ bed + bath + area + year_built + cooling +  
    lot, data = duke_forest)
```

Residuals:

Min	1Q	Median	3Q	Max
-723283	-78169	-1185	85504	383447

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-2.911e+06	1.788e+06	-1.628	0.10703
bed	-1.369e+04	2.593e+04	-0.528	0.59874
bath	4.108e+04	2.466e+04	1.666	0.09928 .
area	1.021e+02	2.313e+01	4.416	2.79e-05 ***
year_built	1.459e+03	9.140e+02	1.597	0.11385
coolingcentral	8.407e+04	3.034e+04	2.771	0.00679 **
lot	3.561e+05	7.594e+04	4.690	9.70e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 145100 on 90 degrees of freedom

Multiple R-squared: 0.6153, Adjusted R-squared: 0.5896

F-statistic: 23.99 on 6 and 90 DF, p-value: < 2.2e-16

Tutorial: Backward Elimination w/ R^2

We had an adjusted R^2 value of 0.5896. We will now make k models, each with one variable missing:

- `mod1_b11 = lm(price ~ bath + area + year_built + cooling + lot, data = duke_forest)`
- `mod1_b12 = lm(price ~ bed + area + year_built + cooling + lot, data = duke_forest)`
- `mod1_b13 = lm(price ~ bed + bath + year_built + cooling + lot, data = duke_forest)`
- `mod1_b14 = lm(price ~ bed + bath + area + cooling + lot, data = duke_forest)`
- `mod1_b15 = lm(price ~ bed + bath + area + year_built + lot, data = duke_forest)`
- `mod1_b16 = lm(price ~ bed + bath + area + year_built + cooling, data = duke_forest)`

Tutorial: Backward Elimination w/ R^2

Using the `summary()` on each of the following models, we get the following results:

	-bed	-bath	-area	-year_built	-cooling	-lot
R^2	0.5929	0.5816	0.5062	0.5826	0.5595	0.4894

Removing the bed variable resulted in the highest adjusted R^2 value among the models that performed better than the baseline model. As such, the new model is the following:

- `mod1_b11 = lm(price ~ bath + area + year_built + cooling + lot, data = duke_forest)`

Tutorial: Backward Elimination w/ R^2

We have an adjusted R^2 value of 0.5929 in the pruned model. We will now make $k - 1$ models, each with one variable missing:

- `mod1_b21 = lm(price ~ area + year_built + cooling + lot, data = duke_forest)`
- `mod1_b22 = lm(price ~ bath + year_built + cooling + lot, data = duke_forest)`
- `mod1_b23 = lm(price ~ bath + area + cooling + lot, data = duke_forest)`
- `mod1_b24 = lm(price ~ bath + area + year_built + lot, data = duke_forest)`
- `mod1_b25 = lm(price ~ bath + area + year_built + cooling, data = duke_forest)`

Tutorial: Backward Elimination w/ R^2

Using the `summary()` on each of the following models, we get the following results:

	-bath	-area	-year_built	-cooling	-lot
R^2	0.5861	0.5100	0.5858	0.5632	0.4932

All of the pruned models result in lower adjusted R^2 values than before. As such, the final pruned model via backward elimination is:

- `mod1_b11 = lm(price ~ bath + area + year_built + cooling + lot, data = duke_forest)`

Tutorial: Forward Selection w/ R^2

We start with the following model fit, which has no predictors:

- `mod1_f0 = lm(price ~ 1, data = duke_forest)`
- `summary(mod1_f0)`

Tutorial: Forward Selection w/ R^2

We will now make k models, each with an addition of one variable:

- `mod1_f11 = lm(price ~ bed, data = duke_forest)`
- `mod1_f12 = lm(price ~ bath, data = duke_forest)`
- `mod1_f13 = lm(price ~ area, data = duke_forest)`
- `mod1_f14 = lm(price ~ year_built, data = duke_forest)`
- `mod1_f15 = lm(price ~ cooling, data = duke_forest)`
- `mod1_f16 = lm(price ~ lot, data = duke_forest)`

Tutorial: Forward Selection w/ R^2

Using the `summary()` on each of the following models, we get the following results:

	+bed	+bath	+area	+year_built	+cooling	+lot
R^2	0.1607	0.3458	0.4449	0.0374	0.0330	0.2777

Including the area variable resulted in the highest adjusted R^2 value among the models that performed better than the baseline model. As such, the new model is the following:

- `mod1_f13 = lm(price ~ area, data = duke_forest)`

Tutorial: Forward Selection w/ R^2

We have an adjusted R^2 value of 0.4449 in the pruned model. We will now make $k - 1$ models, each with one variable missing:

- `mod1_f21 = lm(price ~ area + bed, data = duke_forest)`
- `mod1_f22 = lm(price ~ area + bath, data = duke_forest)`
- `mod1_f23 = lm(price ~ area + year_built, data = duke_forest)`
- `mod1_f24 = lm(price ~ area + cooling, data = duke_forest)`
- `mod1_f25 = lm(price ~ area + lot, data = duke_forest)`

Tutorial: Forward Selection w/ R^2

Using the `summary()` on each of the following models, we get the following results:

	+bed	+bath	+year_built	+cooling	+lot
R^2	0.4404	0.4760	0.4502	0.4849	0.524

Including the `lot` variable resulted in the highest adjusted R^2 value among the models that performed better than the previous model. As such, the new model is the following:

- `mod1_f25 = lm(price ~ area + lot, data = duke_forest)`

We can continue doing this until we get the best-fit model, which has an adjusted R^2 of 0.5929:

- `mod1_f52 = lm(price ~ area + lot + cooling + year_built + bath, data = duke_forest)`

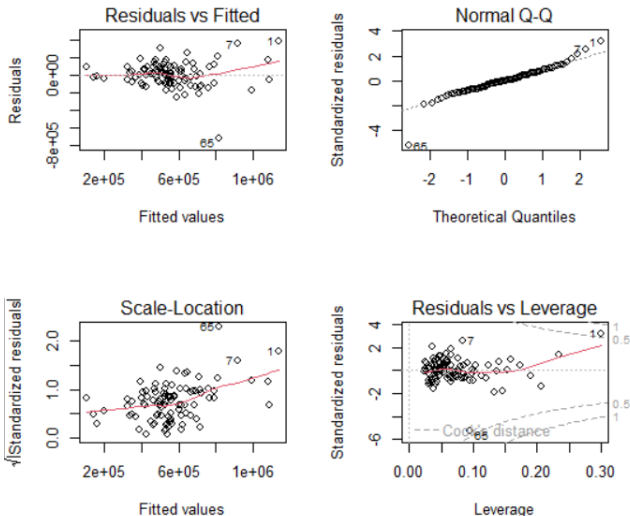
Diagnostics

Recall the four conditions:

- 1 **Linearity**: data shows a linear trend
- 2 **Nearly normal residuals**: residuals should follow approximately a normal distribution
- 3 **Constant variability**: variability around regression line should remain constant throughout all x
- 4 **Independent observations**: should not have some underlying structure (e.g. time series data)

As seen in the last set of slides, we can use the `plot()` function on our model to diagnose these conditions. However, we can dive deeper in the hopes of improving our fit.

Tutorial: Duke Forest Dataset



Diagnostics - Explanatory Variable vs. Response

We can evaluate the residuals against each of the predictor variables as well. To do so, we can use the following code:

- `plot(var, residuals)`

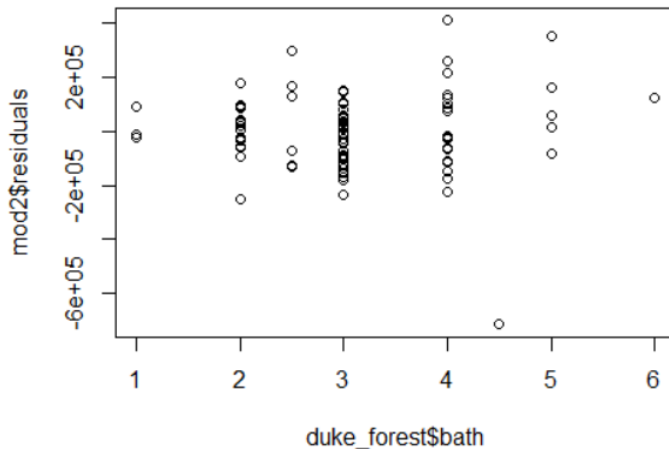
For our dataset, let us use the following model for future analysis:

- `mod2 = lm(price ~ bath + area + year_built + cooling + lot, data = duke_forest)`

We can evaluate the number of bathrooms against the residuals:

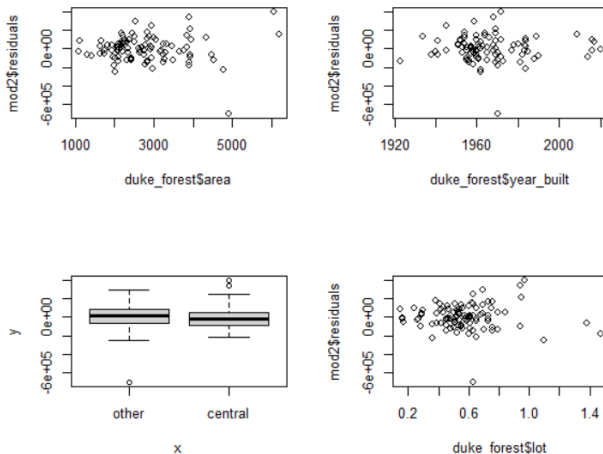
- `plot(duke_forest$bath, mod2$residuals)`

Diagnostics - Explanatory Variable vs. Response



Diagnostics - Explanatory Variable vs. Response

We can do the same with all other explanatory variables:



Aside: Transformations

In some cases, we may see that the residuals (and their spread) vary on the relationship of the predictor variable

- e.g. direction, wider spread of points over time

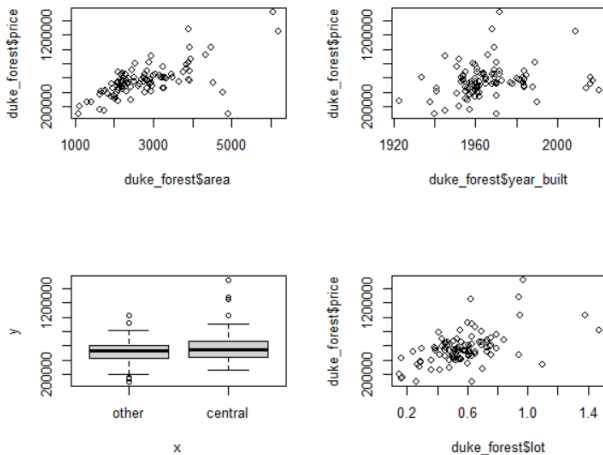
To combat some of these issues, we can apply transformations to the predictor(s) as needed. Some of these transformations include:

- Log transformation ($\log x$)
- Square root transformation ($\sqrt{x}$)
- Inverse transformation ($1/x$)
- Quadratic transformation (x^2)
- Truncation ($x > \text{max} \implies x = \text{max}$, can be *min* as well)

These transformations can be done by adding new variables to our dataset and using these new variables in the refined model. The EDA will provide context to what transformation you should apply.

Tutorial: Duke Forest Dataset - Transformations

If we plot all explanatory variables (except `bath`) against the response variable, we can get the following plots:



Tutorial: Duke Forest Dataset - Transformations

We may be inclined to apply a square root transformation since there appears to be a square root relationship (although not entirely clear). If we apply this transformation, we get the following plot:

- `duke_forest$areasqrt = sqrt(duke_forest$area)`

From here, we can create a new version of our model:

- `mod3 = lm(price ~ bath + areasqrt + year_built + cooling + lot, data = duke_forest)`

The same steps for diagnostics and model selection apply here.

Final Thoughts

“All models are wrong, but some are useful” -George E.P. Box

- No model is ever perfect!
- We report models with conditions that will, for the most part, be slightly violated
 - ▶ We just need to acknowledge the violations while reporting
- If conditions are clearly violated, do not report results and consider a new model

Interpretation and Hypothesis Testing

- Procedures for interpretation and hypothesis testing remain the same
- When doing interpretation on a β coefficient:
 - ▶ We assume all other β coefficients are constant when talking about the change in the β coefficient of interest

Prediction

- When creating the dataset for prediction in R, make sure to specify all variables and their values, e.g.
 - ▶ `new_data <- data.frame(bath = c(3, 1), area = c(2000, 1000), year_built = c(1980, 2000), cooling = c(1, 1), lot = c(0.4, 0.5))`
 - ▶ `predict(model, data)`

For Friday!

- Homework #4 has been assigned
 - ▶ Involves textbook problems and an R question
 - ▶ Due on Sunday at 11:59pm
- Checkpoint #3 during Friday's class
 - ▶ 15 minute warm-up quiz at beginning of class